HOSPITAL MANAGEMENT SYSTEM

Relational Database Design and Implementation using MySQL

Name: Dipson Upadhyay

Class: Grade 12 Computer Science

Database: Hospital_Management

1. Introduction

This project presents the design and implementation of a Hospital Management System using MySQL. The purpose of the database is to organize information about patients, doctors, appointments and medical records in a structured and connected form.

The project follows the basic stages of relational database development: identifying entities, defining primary and foreign keys, choosing suitable data types and constraints, creating the tables, inserting sample records and performing SQL queries to retrieve and modify information.

The database contains four related tables: patients, doctors, appointments and medical_records.

2. Entity-Relationship (ER) Diagram

The ER diagram shows the main entities and relationships formed through primary and foreign keys.

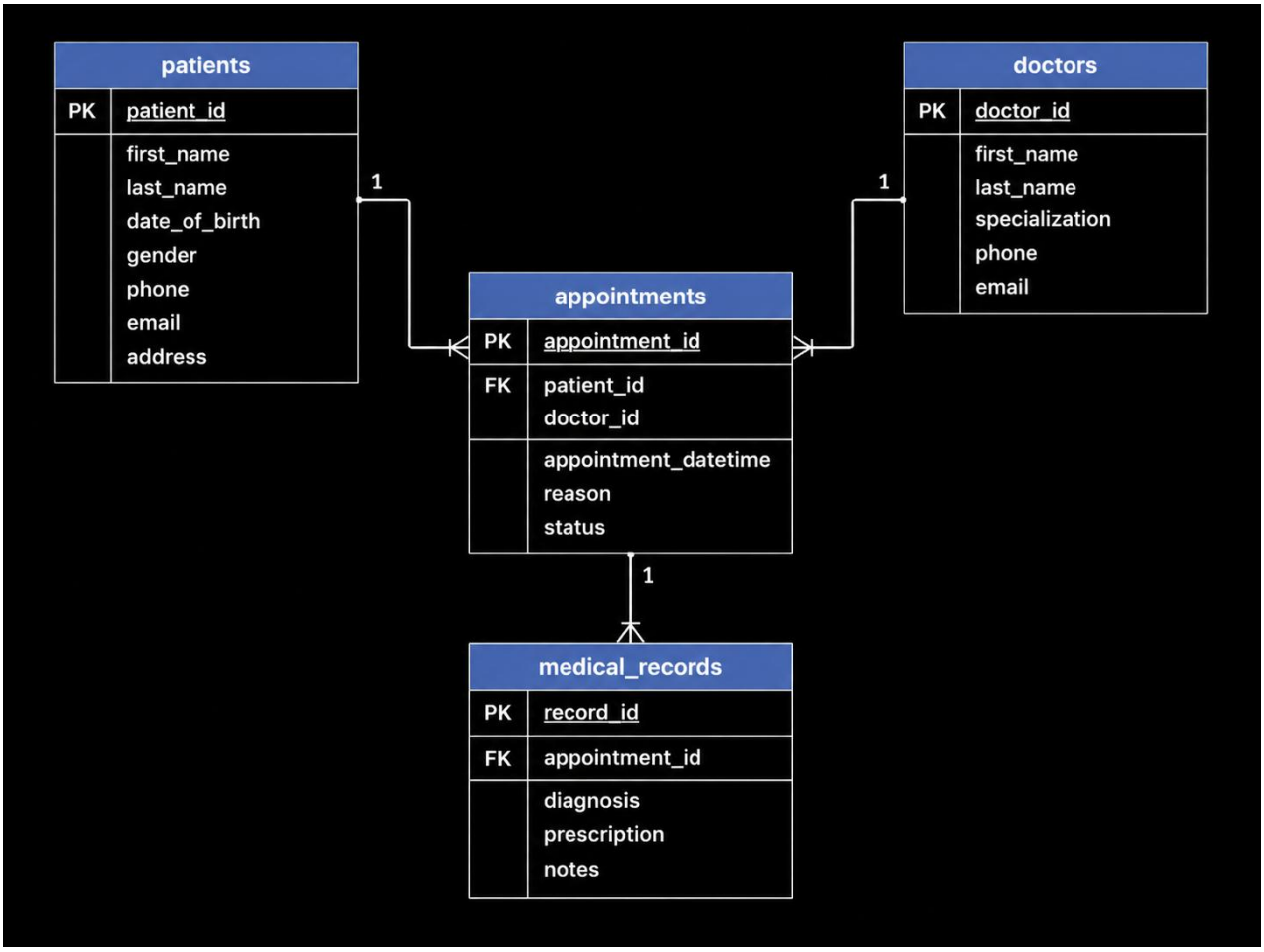


Figure 1: Entity-Relationship diagram of the Hospital Management System.

A patient can have multiple appointments and medical records, while a doctor can be associated with multiple appointments and medical records.

3. Table Structure, Data Types and Constraints

The following tables implement the ER design. The data types and constraints are selected to maintain valid and consistent records.

Table: patients

Column Name	Data Type	Constraints	Description
patient_id	INT	PRIMARY KEY	Unique identifier for each patient.
first_name	VARCHAR(50)	NOT NULL	Patient's first name.
last_name	VARCHAR(50)	NOT NULL	Patient's last name.
email	VARCHAR(100)	NOT NULL, UNIQUE	Patient's email address.
date_of_birth	DATE	NOT NULL	Patient's date of birth.
blood_group	VARCHAR(5)	NOT NULL	Patient's blood group.
status	VARCHAR(20)	DEFAULT 'Active', CHECK	Current patient status.

Table: doctors

Column Name	Data Type	Constraints	Description
doctor_id	INT	PRIMARY KEY	Unique identifier for each doctor.
first_name	VARCHAR(50)	NOT NULL	Doctor's first name.
last_name	VARCHAR(50)	NOT NULL	Doctor's last name.
email	VARCHAR(100)	NOT NULL, UNIQUE	Doctor's email address.
specialization	VARCHAR(60)	NOT NULL	Doctor's medical

			specialization.
consultation_fee	DECIMAL(8,2)	NOT NULL, CHECK	Consultation fee charged by the doctor.

Table: appointments

Column Name	Data Type	Constraints	Description
appointment_id	INT	PRIMARY KEY	Unique identifier for each appointment.
patient_id	INT	NOT NULL, FOREIGN KEY	References patients(patient_id).
doctor_id	INT	NOT NULL, FOREIGN KEY	References doctors(doctor_id).
appointment_date	DATE	NOT NULL	Date of the appointment.
appointment_time	TIME	NOT NULL	Time of the appointment.
reason	VARCHAR(150)	NOT NULL	Reason for the appointment.
status	VARCHAR(20)	DEFAULT 'Scheduled', CHECK	Appointment status.

Table: medical_records

Column	Data Type	Constraints	Description

Name			
record_id	INT	PRIMARY KEY	Unique identifier for each medical record.
patient_id	INT	NOT NULL, FOREIGN KEY	References patients(patient_id).
doctor_id	INT	NOT NULL, FOREIGN KEY	References doctors(doctor_id).
record_date	DATE	NOT NULL	Date of the medical record.
diagnosis	VARCHAR(150)	NOT NULL	Diagnosis recorded by the doctor.
treatment	VARCHAR(200)	NOT NULL	Treatment or advice recorded.

4. Data Implementation (SQL)

The SQL implementation follows the database design. The database and tables are created first, followed by sample data insertion.

4.1 Basic Database and Table Creation

```
DROP DATABASE IF EXISTS hospital_management;

CREATE DATABASE hospital_management;
USE hospital_management;

CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
```

```
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    date_of_birth DATE NOT NULL,
    blood_group VARCHAR(5) NOT NULL,
    status VARCHAR(20) DEFAULT 'Active',
    CHECK (status IN ('Active', 'Inactive'))
);

CREATE TABLE doctors (
    doctor_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    specialization VARCHAR(60) NOT NULL,
    consultation_fee DECIMAL(8,2) NOT NULL,
    CHECK (consultation_fee > 0)
);

CREATE TABLE appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    reason VARCHAR(150) NOT NULL,
    status VARCHAR(20) DEFAULT 'Scheduled',
    CHECK (status IN ('Scheduled', 'Completed',
'Cancelled')),
    FOREIGN KEY (patient_id) REFERENCES
patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES
doctors(doctor_id)
);

CREATE TABLE medical_records (
    record_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
```

```
    record_date DATE NOT NULL,
    diagnosis VARCHAR(150) NOT NULL,
    treatment VARCHAR(200) NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES
patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES
doctors(doctor_id)
);
```

4.2 Inserting Values

Ten sample records are inserted into each table.

```
INSERT INTO patients
(patient_id, first_name, last_name, email, date_of_birth,
blood_group, status)
VALUES
(1, 'Oliver', 'Carter', 'oliver.carter@example.com', '2004-03-14',
'A+', 'Active'),
(2, 'Emma', 'Wilson', 'emma.wilson@example.com', '1998-07-21', 'B+',
'Active'),
(3, 'Noah', 'Bennett', 'noah.bennett@example.com', '2001-01-10',
'O+', 'Active'),
(4, 'Olivia', 'Mitchell', 'olivia.mitchell@example.com', '1995-11-
05', 'AB+', 'Active'),
(5, 'Ethan', 'Parker', 'ethan.parker@example.com', '1989-04-18', 'O-
', 'Active'),
(6, 'Sophia', 'Turner', 'sophia.turner@example.com', '2000-09-12',
'A-', 'Active'),
(7, 'Lucas', 'Anderson', 'lucas.anderson@example.com', '1992-06-25',
'B-', 'Active'),
(8, 'Ava', 'Morgan', 'ava.morgan@example.com', '1997-03-30', 'A+',
'Active'),
(9, 'James', 'Cooper', 'james.cooper@example.com', '1985-08-16',
'O+', 'Active'),
(10, 'Isabella', 'Reed', 'isabella.reed@example.com', '1990-12-09',
'AB-', 'Inactive');

INSERT INTO doctors
(doctor_id, first_name, last_name, email, specialization,
consultation_fee)
VALUES
(1, 'William', 'Harris', 'william.harris@hospital.com', 'Cardiology',
```

```
80.00),
(2, 'Emily', 'Robinson', 'emily.robinson@hospital.com',
'Dermatology', 65.00),
(3, 'Daniel', 'Clark', 'daniel.clark@hospital.com', 'Neurology',
90.00),
(4, 'Sarah', 'Lewis', 'sarah.lewis@hospital.com', 'Pediatrics',
60.00),
(5, 'Michael', 'Walker', 'michael.walker@hospital.com',
'Orthopedics', 75.00),
(6, 'Jessica', 'Hall', 'jessica.hall@hospital.com', 'Gynecology',
70.00),
(7, 'Thomas', 'Young', 'thomas.young@hospital.com', 'General
Medicine', 50.00),
(8, 'Laura', 'King', 'laura.king@hospital.com', 'Ophthalmology',
55.00),
(9, 'Robert', 'Scott', 'robert.scott@hospital.com', 'ENT', 60.00),
(10, 'Anna', 'Green', 'anna.green@hospital.com', 'Psychiatry',
85.00);

INSERT INTO appointments
(appointment_id, patient_id, doctor_id, appointment_date,
appointment_time, reason, status)
VALUES
(1, 1, 1, '2026-08-01', '09:00:00', 'Chest discomfort', 'Completed'),
(2, 2, 2, '2026-08-02', '10:30:00', 'Skin irritation', 'Completed'),
(3, 3, 3, '2026-08-03', '11:00:00', 'Frequent headaches',
'Completed'),
(4, 4, 5, '2026-08-04', '14:00:00', 'Knee pain', 'Completed'),
(5, 5, 7, '2026-08-05', '09:30:00', 'Routine checkup', 'Completed'),
(6, 6, 4, '2026-08-06', '13:00:00', 'Child health check',
'Completed'),
(7, 7, 8, '2026-08-07', '15:30:00', 'Blurred vision', 'Scheduled'),
(8, 8, 9, '2026-08-08', '10:00:00', 'Ear pain', 'Scheduled'),
(9, 9, 1, '2026-08-09', '12:00:00', 'High blood pressure',
'Completed'),
(10, 10, 10, '2026-08-10', '16:00:00', 'Sleep problems',
'Cancelled');

INSERT INTO medical_records
(record_id, patient_id, doctor_id, record_date, diagnosis, treatment)
VALUES
(1, 1, 1, '2026-08-01', 'Mild hypertension', 'Blood pressure
monitoring'),
(2, 2, 2, '2026-08-02', 'Skin allergy', 'Antihistamine and skin
```

```
cream'),
(3, 3, 3, '2026-08-03', 'Migraine', 'Rest and prescribed
medication'),
(4, 4, 5, '2026-08-04', 'Knee strain', 'Physiotherapy and rest'),
(5, 5, 7, '2026-08-05', 'Vitamin deficiency', 'Dietary supplements'),
(6, 6, 4, '2026-08-06', 'Routine examination', 'No major treatment
required'),
(7, 7, 8, '2026-08-07', 'Vision difficulty', 'Eye examination and
glasses'),
(8, 8, 9, '2026-08-08', 'Ear infection', 'Prescribed antibiotics'),
(9, 9, 1, '2026-08-09', 'Hypertension', 'Medication and regular
monitoring'),
(10, 10, 10, '2026-08-10', 'Insomnia', 'Sleep management advice');
```

5. Data Querying

5.1 Basic Retrieval

Shows active patients, sorts them alphabetically by last name and displays the first five records.

```
SELECT *
FROM patients
WHERE status = 'Active'
ORDER BY last_name ASC
LIMIT 5;
```

5.2 Aggregate Function, GROUP BY and HAVING

Counts appointments for each doctor and calculates the percentage of completed appointments.

```
SELECT
    doctor_id,
    COUNT(*) AS total_appointments,
    AVG(
        CASE
            WHEN status = 'Completed' THEN 1
            ELSE 0
        END
```

```
    ) * 100 AS completion_rate
FROM appointments
GROUP BY doctor_id
HAVING COUNT(*) >= 1
ORDER BY completion_rate DESC;
```

5.3 Subquery

Finds doctors whose consultation fee is above the average consultation fee.

```
SELECT
    doctor_id,
    first_name,
    last_name,
    consultation_fee
FROM doctors
WHERE consultation_fee > (
    SELECT AVG(consultation_fee)
    FROM doctors
)
ORDER BY consultation_fee DESC;
```

5.4 UPDATE

Changes the status of appointment 7 to Completed using a specific WHERE condition.

```
UPDATE appointments
SET status = 'Completed'
WHERE appointment_id = 7;

SELECT *
FROM appointments
WHERE appointment_id = 7;
```

5.5 DELETE

Deletes appointment 10 using its primary key so that an unintended set of records is not removed.

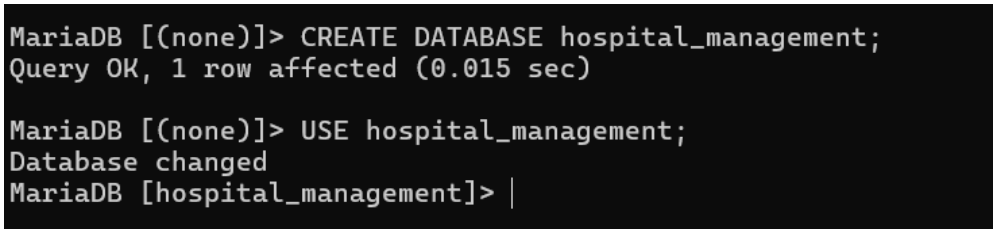
```
DELETE FROM appointments
WHERE appointment_id = 10;

SELECT *
FROM appointments
WHERE appointment_id = 10;
```

6. Proof of Successful Execution

Insert actual screenshots from MySQL Workbench or the MySQL terminal in this section. Screenshots should clearly show successful execution and output.

Figure 2: Database and tables created successfully.



```
MariaDB [hospital_management]> CREATE TABLE patients (
    ->     patient_id INT PRIMARY KEY,
    ->     first_name VARCHAR(50) NOT NULL,
    ->     last_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) NOT NULL UNIQUE,
    ->     date_of_birth DATE NOT NULL,
    ->     blood_group VARCHAR(5) NOT NULL,
    ->     status VARCHAR(20) DEFAULT 'Active',
    ->     CHECK (status IN ('Active', 'Inactive'))
    -> );
Query OK, 0 rows affected (0.030 sec)

MariaDB [hospital_management]> CREATE TABLE doctors (
    ->     doctor_id INT PRIMARY KEY,
    ->     first_name VARCHAR(50) NOT NULL,
    ->     last_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) NOT NULL UNIQUE,
    ->     specialization VARCHAR(60) NOT NULL,
    ->     consultation_fee DECIMAL(8,2) NOT NULL,
    ->     CHECK (consultation_fee > 0)
    -> );
Query OK, 0 rows affected (0.015 sec)

MariaDB [hospital_management]> CREATE TABLE appointments (
    ->     appointment_id INT PRIMARY KEY,
    ->     patient_id INT NOT NULL,
    ->     doctor_id INT NOT NULL,
    ->     appointment_date DATE NOT NULL,
    ->     appointment_time TIME NOT NULL,
    ->     reason VARCHAR(150) NOT NULL,
    ->     status VARCHAR(20) DEFAULT 'Scheduled',
    ->     CHECK (status IN ('Scheduled', 'Completed', 'Cancelled')),
    ->     FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    ->     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
    -> );
Query OK, 0 rows affected (0.016 sec)

MariaDB [hospital_management]>
MariaDB [hospital_management]> CREATE TABLE medical_records (
    ->     record_id INT PRIMARY KEY,
    ->     patient_id INT NOT NULL,
    ->     doctor_id INT NOT NULL,
    ->     record_date DATE NOT NULL,
    ->     diagnosis VARCHAR(150) NOT NULL,
    ->     treatment VARCHAR(200) NOT NULL,
    ->     FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    ->     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
    -> );
Query OK, 0 rows affected (0.016 sec)

MariaDB [hospital_management]>
```

Figure 3: Sample records inserted into the database.

```
MariaDB [hospital_management]> INSERT INTO patients
    -> (patient_id, first_name, last_name, email, date_of_birth, blood_group, status)
    -> VALUES
    -> (1, 'Oliver', 'Carter', 'oliver.carter@example.com', '2004-03-14', 'A+', 'Active'),
    -> (2, 'Emma', 'Wilson', 'emma.wilson@example.com', '1998-07-21', 'B+', 'Active'),
    -> (3, 'Noah', 'Bennett', 'noah.bennett@example.com', '2001-01-10', 'O+', 'Active'),
    -> (4, 'Olivia', 'Mitchell', 'olivia.mitchell@example.com', '1995-11-05', 'AB+', 'Active'),
    -> (5, 'Ethan', 'Parker', 'ethan.parker@example.com', '1989-04-18', 'O-', 'Active'),
    -> (6, 'Sophia', 'Turner', 'sophia.turner@example.com', '2000-09-12', 'A-', 'Active'),
    -> (7, 'Lucas', 'Anderson', 'lucas.anderson@example.com', '1992-06-25', 'B-', 'Active'),
    -> (8, 'Ava', 'Morgan', 'ava.morgan@example.com', '1997-03-30', 'A+', 'Active'),
    -> (9, 'James', 'Cooper', 'james.cooper@example.com', '1985-08-16', 'O+', 'Active'),
    -> (10, 'Isabella', 'Reed', 'isabella.reed@example.com', '1990-12-09', 'AB-', 'Inactive');
Query OK, 10 rows affected (0.031 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
MariaDB [hospital_management]> INSERT INTO doctors
    -> (doctor_id, first_name, last_name, email, specialization, consultation_fee)
    -> VALUES
    -> (1, 'William', 'Harris', 'william.harris@hospital.com', 'Cardiology', 80.00),
    -> (2, 'Emily', 'Robinson', 'emily.robinson@hospital.com', 'Dermatology', 65.00),
    -> (3, 'Daniel', 'Clark', 'daniel.clark@hospital.com', 'Neurology', 90.00),
    -> (4, 'Sarah', 'Lewis', 'sarah.lewis@hospital.com', 'Pediatrics', 60.00),
    -> (5, 'Michael', 'Walker', 'michael.walker@hospital.com', 'Orthopedics', 75.00),
    -> (6, 'Jessica', 'Hall', 'jessica.hall@hospital.com', 'Gynecology', 70.00),
    -> (7, 'Thomas', 'Young', 'thomas.young@hospital.com', 'General Medicine', 50.00),
    -> (8, 'Laura', 'King', 'laura.king@hospital.com', 'Ophthalmology', 55.00),
    -> (9, 'Robert', 'Scott', 'robert.scott@hospital.com', 'ENT', 60.00),
    -> (10, 'Anna', 'Green', 'anna.green@hospital.com', 'Psychiatry', 85.00);
Query OK, 10 rows affected (0.006 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
MariaDB [hospital_management]> INSERT INTO appointments
    -> (appointment_id, patient_id, doctor_id, appointment_date, appointment_time, reason, st
    -> VALUES
    -> (1, 1, 1, '2026-08-01', '09:00:00', 'Chest discomfort', 'Completed'),
    -> (2, 2, 2, '2026-08-02', '10:30:00', 'Skin irritation', 'Completed'),
    -> (3, 3, 3, '2026-08-03', '11:00:00', 'Frequent headaches', 'Completed'),
    -> (4, 4, 5, '2026-08-04', '14:00:00', 'Knee pain', 'Completed'),
    -> (5, 5, 7, '2026-08-05', '09:30:00', 'Routine checkup', 'Completed'),
    -> (6, 6, 4, '2026-08-06', '13:00:00', 'Child health check', 'Completed'),
    -> (7, 7, 8, '2026-08-07', '15:30:00', 'Blurred vision', 'Scheduled'),
    -> (8, 8, 9, '2026-08-08', '10:00:00', 'Ear pain', 'Scheduled'),
    -> (9, 9, 1, '2026-08-09', '12:00:00', 'High blood pressure', 'Completed'),
    -> (10, 10, 10, '2026-08-10', '16:00:00', 'Sleep problems', 'Cancelled');
Query OK, 10 rows affected (0.009 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
MariaDB [hospital_management]> INSERT INTO medical_records
    -> (record_id, patient_id, doctor_id, record_date, diagnosis, treatment)
    -> VALUES
    -> (1, 1, 1, '2026-08-01', 'Mild hypertension', 'Blood pressure monitoring'),
    -> (2, 2, 2, '2026-08-02', 'Skin allergy', 'Antihistamine and skin cream'),
    -> (3, 3, 3, '2026-08-03', 'Migraine', 'Rest and prescribed medication'),
    -> (4, 4, 5, '2026-08-04', 'Knee strain', 'Physiotherapy and rest'),
    -> (5, 5, 7, '2026-08-05', 'Vitamin deficiency', 'Dietary supplements'),
    -> (6, 6, 4, '2026-08-06', 'Routine examination', 'No major treatment required'),
    -> (7, 7, 8, '2026-08-07', 'Vision difficulty', 'Eye examination and glasses'),
    -> (8, 8, 9, '2026-08-08', 'Ear infection', 'Prescribed antibiotics'),
    -> (9, 9, 1, '2026-08-09', 'Hypertension', 'Medication and regular monitoring'),
    -> (10, 10, 10, '2026-08-10', 'Insomnia', 'Sleep management advice');
Query OK, 10 rows affected (0.007 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Figure 4: Basic retrieval query output.

```
MariaDB [hospital_management]> SELECT *
    -> FROM patients
    -> WHERE status = 'Active'
    -> ORDER BY last_name ASC
    -> LIMIT 5;
+------------+------------+-----------+---------------------------+---------------+-------------+--------+
| patient_id | first_name | last_name | email                     | date_of_birth | blood_group | status |
+------------+------------+-----------+---------------------------+---------------+-------------+--------+
|          7 | Lucas      | Anderson  | lucas.anderson@example.com | 1992-06-25    | B-          | Active |
|          3 | Noah       | Bennett   | noah.bennett@example.com   | 2001-01-10    | O+          | Active |
|          1 | Oliver     | Carter    | oliver.carter@example.com  | 2004-03-14    | A+          | Active |
|          9 | James      | Cooper    | james.cooper@example.com   | 1985-08-16    | O+          | Active |
|          4 | Olivia     | Mitchell  | olivia.mitchell@example.com | 1995-11-05    | AB+         | Active |
+------------+------------+-----------+---------------------------+---------------+-------------+--------+
5 rows in set (0.008 sec)
```

Figure 5: Aggregate function with GROUP BY and HAVING output.

```
MariaDB [hospital_management]> SELECT
    ->     doctor_id,
    ->     COUNT(*) AS total_appointments,
    ->     AVG(
    ->         CASE
    ->             WHEN status = 'Completed' THEN 1
    ->             ELSE 0
    ->         END
    ->     ) * 100 AS completion_rate
    -> FROM appointments
    -> GROUP BY doctor_id
    -> HAVING COUNT(*) >= 1
    -> ORDER BY completion_rate DESC;
+-----------+--------------------+-----------------+
| doctor_id | total_appointments | completion_rate |
+-----------+--------------------+-----------------+
|         3 |                  1 |         99.9900 |
|         4 |                  1 |         99.9900 |
|         5 |                  1 |         99.9900 |
|         7 |                  1 |         99.9900 |
|         1 |                  2 |         99.9900 |
|         2 |                  1 |         99.9900 |
|         8 |                  1 |          0.0000 |
|         9 |                  1 |          0.0000 |
|        10 |                  1 |          0.0000 |
+-----------+--------------------+-----------------+
9 rows in set (0.006 sec)
```

Figure 6: Subquery output.

```
MariaDB [hospital_management]> SELECT
    ->     doctor_id,
    ->     first_name,
    ->     last_name,
    ->     consultation_fee
    -> FROM doctors
    -> WHERE consultation_fee > (
    ->     SELECT AVG(consultation_fee)
    ->     FROM doctors
    -> )
    -> ORDER BY consultation_fee DESC;
+-----------+------------+-----------+------------------+
| doctor_id | first_name | last_name | consultation_fee |
+-----------+------------+-----------+------------------+
|         3 | Daniel     | Clark     |            90.00 |
|        10 | Anna       | Green     |            85.00 |
|         1 | William    | Harris    |            80.00 |
|         5 | Michael    | Walker    |            75.00 |
|         6 | Jessica    | Hall      |            70.00 |
+-----------+------------+-----------+------------------+
5 rows in set (0.002 sec)
```

Figure 7: UPDATE and DELETE execution/output.

```
MariaDB [hospital_management]> UPDATE appointments
    -> SET status = 'Completed'
    -> WHERE appointment_id = 7;
Query OK, 1 row affected (0.007 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
MariaDB [hospital_management]> DELETE FROM appointments
    -> WHERE appointment_id = 10;
Query OK, 1 row affected (0.002 sec)

MariaDB [hospital_management]>
MariaDB [hospital_management]> SELECT *
    -> FROM appointments
    -> WHERE appointment_id = 10;
Empty set (0.000 sec)
```

7. Conclusion

The Hospital Management System successfully demonstrates the design, implementation and querying of a relational database using MySQL. The four tables provide a clear structure for storing patient, doctor, appointment and medical record information.

The project includes primary and foreign keys, suitable data types and integrity constraints. It also demonstrates basic retrieval, aggregate functions, grouping and filtering, a subquery, an INNER JOIN, a LEFT JOIN, UPDATE and DELETE operations.

The completed database provides a simple but practical foundation for managing hospital information while demonstrating the core relational database concepts required in the project.

8. Reflection

Working on this project helped me understand how separate tables can be connected to represent a real-world system. At first, database design looked mainly like writing CREATE TABLE commands, but designing the relationships and deciding which information belongs in each table made the structure much clearer.

I also understood the importance of primary keys and foreign keys because they prevent unrelated records from being connected incorrectly. Writing different queries made me more comfortable with SQL, especially when combining information from multiple tables and filtering grouped results.

Overall, this project gave me a better understanding of how a relational database is planned before it is implemented. It also showed me that good table structure and data integrity are just as important as the queries used to retrieve the data.